Wildfire Detection CNN

Next-Day Wildfire Spread Prediction

using Satellite Imagery, Terrain & Climate Data

Dataset
Kaggle TFRecord

Model
Attention U-Net

Output
Fire Spread Mask

March 2026

Agenda

Data & Exploration

01 Problem Statement

02 Dataset & File Format

03 TFRecord Structure

04 The 12 Input Features

05 EDA & Visualisation

06 Preprocessing

Model & Results

07 Model Architecture

08 Mathematics Used

09 Training Pipeline

10 Validation & Testing

11 Web Application

12 Use Cases & Advantages

01. Problem Statement

The Challenge

Wildfires are increasing in frequency and severity. Early prediction of where a fire will spread **next day** can save lives, property, and ecosystems.

Goal: Given today's satellite + climate snapshot at a location, predict the **binary fire mask for tomorrow**.

Semantic Segmentation task:

- Input: $12 \times 64 \times 64$ multi-channel patch
- Output: $1 \times 64 \times 64$ binary mask ($1 =$ fire, $0 =$ no fire)

Key Numbers

Patch size	64×64 px
Channels	12
Task	Binary Seg.
Class ratio	$\sim$2% fire
Source	Landsat-8

Real-World Scale

Each pixel $\approx$ 375 m on a side.
One 64×64 patch covers ≈ 1.5 km^2 of terrain.
Correct prediction gives 24 h advance warning.

02. Dataset — Next Day Wildfire Spread (Kaggle)

Source: Kaggle — `fantineh/next-day-wildfire-spread`

Originally: Huot et al. (2022), Google Earth Engine

What you download:

- Folder structure with 3 splits: `train/`, `eval/`, `test/`
- Each split: multiple **.tfrecord shard files**
- e.g. `train-00001-of-00020.tfrecord`

Split Sizes

Split	Samples	Approx Fire
Train	∼18,000	∼2%
Eval	∼2,200	∼2%

Why TFRecord?

Binary serialisation (Protocol Buffers):

- Compact binary storage
- Fast sequential reads
- Native to TensorFlow pipelines
- Each shard ≈ 100–200 MB
- Full dataset ≈ 4–5 GB

Per-Sample Storage

13 feature planes × 4096 float32
= 13 × 4096 × 4 bytes
∼ 214 KB per sample (uncompressed)

03. TFRecord File Format — How Data is Stored

Structure of one TFRecord example:

Each record stores **13 flat arrays**, each of length $64 \times 64 = 4096$ float32 values.

Parsing Schema

```
feat:  FixedLenFeature([4096], float32)
```

After parsing: reshape $[4096] \rightarrow [64, 64]$

Extraction steps:

TFRecord Example (one sample)

```
NDVI: [4096 float32]
elevation:  [4096 float32]
th:  [4096 float32]
vs:  [4096 float32]
tmmn:  [4096 float32]
tmmx:  [4096 float32]
sph:  [4096 float32]
pr:  [4096 float32]
pdsi:  [4096 float32]
erc:  [4096 float32]
population:  [4096 float32]
PrevFireMask:  [4096 float32]
FireMask:  [4096 float32]
```

04. The 12 Input Features — What They Mean

Ch	Feature	Full Name	Source / Unit
0	`NDVI`	Normalised Difference Vegetation Index	Landsat-8 / $[-1, 1]$
1	`elevation`	Terrain Elevation	SRTM / metres
2	`th`	Wind Direction	GRIDMET / degrees
3	`vs`	Wind Speed	GRIDMET / m/s
4	`tmmn`	Min Daily Temperature	GRIDMET / Kelvin
5	`tmmx`	Max Daily Temperature	GRIDMET / Kelvin
6	`sph`	Specific Humidity	GRIDMET / kg/kg
7	`pr`	Precipitation	GRIDMET / mm
8	`pdsi`	Palmer Drought Severity Index	GRIDMET / index
9	`erc`	Energy Release Component	GRIDMET / index
10	`population`	Population Density	GPWv4 / persons/km^2
11	`PrevFireMask`	Previous Day Fire Mask	MODIS / binary
L	`FireMask`	Next Day Fire Mask (label)	MODIS / $\{-1, 0, 1\}$

$-1 =$ unknown pixel (masked in loss), $0 =$ no fire, $1 =$ active fire

05. Exploratory Data Analysis (EDA)

Class Imbalance

Fire events are rare — only ∼2% of pixels are fire.

Fire vs No-Fire Pixels

No-Fire	∼98%
Fire	∼2%
Imbalance ratio	≈49:1

Key EDA Findings:

- **PrevFireMask** highest correlation with fire
- High **ERC** + low **NDVI** = high risk
- **Elevation** affects spread direction
- **Wind speed/direction** critical for propagation

Channel Statistics (approx)

Feature	Mean	Std
NDVI	0.30	0.15
Elevation(m)	1200	800
Wind Dir.	220.0	50.0
Wind Speed	3.5	1.8
Min Temp (K)	280.0	12.0
Max Temp (K)	295.0	12.0
Humidity	0.006	0.004
ERC	40.0	25.0
Population	100	500
PrevFire	0.02	0.14

Wide range across channels motivates per-channel Z-score normalisation.

06. Preprocessing & Data Cleaning

1. Unknown Pixel Masking

Label values: $\{-1, 0, 1\}$. -1 = sensor gap / cloud cover (unknown).

Weight mask:

$$w_i = \begin{cases} 0 & \text{if } y_i = -1 \\ 1 & \text{otherwise} \end{cases}$$

Unknown pixels are **excluded from loss**.

2. Z-Score Normalisation

Per-channel using training statistics:

$$\hat{x}_c = \frac{x_c - \mu_c}{\sigma_c + \varepsilon}, \quad \varepsilon = 10^{-6}$$

3. NaN Handling

4. Data Augmentation (train)

Applied on-the-fly, consistently to x, label, and weight:

- Horizontal flip (p=0.5)
- Vertical flip (p=0.5)
- 90/180/270 degree rotation (random)

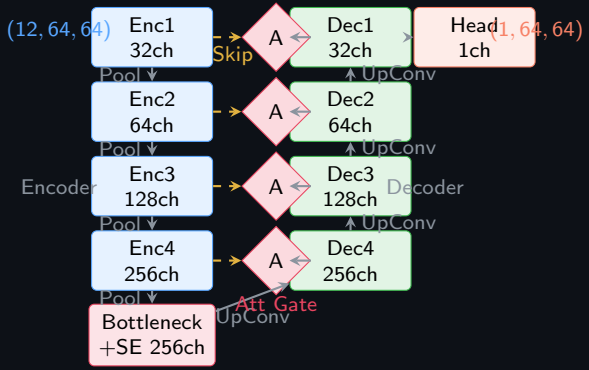
Fire patches are spatially invariant — all augmentations are valid.

Class-Weight for Imbalance

$$w^+ = \frac{N_{\text{neg}}}{N_{\text{pos}}} \approx 49$$

Applied as `pos_weight` in BCE loss to boost

07. Model Architecture — Attention U-Net



Attention gates (A) filter skip connections so the decoder focuses on relevant spatial regions. UNetLite uses base_filters=32 ($\approx$ 1.9M params).

07 (cont.) Architecture Building Blocks

ConvBlock

Two consecutive:
3×3 Conv $\rightarrow$ BN $\rightarrow$ ReLU
3×3 Conv $\rightarrow$ BN $\rightarrow$ ReLU

padding=1 preserves spatial size.
Parameters: $2(C_{in}C_{out} \cdot 9)$

Encoder Stages (Lite)

Stage	Channels
Input	12
Enc1	$12 \rightarrow 32$
Enc2	$32 \rightarrow 64$
Enc3	$64 \rightarrow 128$
Enc4	$128 \rightarrow 256$

Attention Gate

$g' = \mathrm{BN}(W_g \cdot g)$
$s' = \mathrm{BN}(W_s \cdot s)$
$\alpha = \sigma(\psi(\mathrm{ReLU}(g' + s')))$
Output: $\tilde{s} = s \odot \alpha$

g = gating signal (from decoder above)
s = skip feature (from encoder)
$\alpha \in [0, 1]$ = spatial attention map

SE Block

$z = \mathrm{GAP}(F)$
$\hat{z} = \sigma(W_2\,\mathrm{ReLU}(W_1 z))$
$\hat{F} = F \odot \hat{z}$

Decoder Stages (Lite)

Stage	Channels
Up4	256
Dec4	$512 \rightarrow 256$
Up3	256
Dec3	$256 \rightarrow 128$
Up2	128
Dec2	$128 \rightarrow 64$
Up1	64
Dec1	$64 \rightarrow 32$

channels after concat with attention skip

Head

1×1 conv $\rightarrow$ raw logits
Final: $\sigma(\hat{y}) \in [0, 1]$

08. Mathematics — Loss Function

Masked BCE + Dice Loss

Total loss:

$$\mathcal{L} = \lambda_{\text{BCE}} \cdot \mathcal{L}_{\text{BCE}} + \lambda_{\text{Dice}} \cdot \mathcal{L}_{\text{Dice}}$$

with $\lambda_{\text{BCE}} = \lambda_{\text{Dice}} = 0.5$.

Masked BCE:

$$\mathcal{L}_{\text{BCE}} = -\frac{1}{\sum w_i} \sum_i w_i \left[w^+ y_i \log \hat{p}_i + (1-y_i) \log(1-\hat{p}_i) \right]$$

Masked Dice:

$$\mathcal{L}_{\text{Dice}} = 1 - \frac{2 \sum_i w_i \hat{p}_i y_i + 1}{\sum_i w_i \hat{p}_i + \sum_i w_i y_i + 1}$$

Sigmoid: $\hat{p}_i = \sigma(\hat{y}_i) = \frac{1}{\ldots}$

Why Dice?

BCE alone with 49:1 imbalance drives the model to predict "no fire" everywhere.
Dice loss directly maximises prediction-label overlap.

Metrics:

$$\text{IoU} = \frac{TP}{TP + FP + FN}$$

$$\text{F1} = \frac{2 \cdot TP}{2 \cdot TP + FP + FN}$$

$$\text{Precision} = \frac{TP}{TP + FP}, \quad \text{Recall} = \frac{TP}{TP + FN}$$

Positive Class Weight

08 (cont.) Mathematics — Convolutions & Optimiser

2D Convolution

For input $X \in \mathbb{R}^{C_{in} \times H \times W}$, kernel W:

$$(X * W)_{c,h,w} = \sum_{c'} \sum_{i,j} W_{c,c',i,j} \cdot X_{c',h+i,w+j}$$

Padding=1, kernel 3×3 preserves spatial size.

Batch Normalisation

$$\hat{x}_i = \frac{x_i - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \varepsilon}} \cdot \gamma + \beta$$

γ, β learnable per-channel scale & shift.

Transposed Conv (upsampling)

2×2 transposed conv, stride=2:
$H \times W \;\rightarrow\; 2H \times 2W$

Kaiming Init:

$$W \sim \mathcal{N}\left(0, \sqrt{\frac{2}{k^2 C_{in}}}\right)$$

Prevents gradient vanishing with ReLU.

AdamW Optimiser:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$$

$$\theta_t = \theta_{t-1} - \alpha \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \varepsilon} - \alpha \lambda \theta_{t-1}$$

λ = weight decay (decoupled regularisation).

Cosine LR Annealing:

$$\eta_t = \frac{1}{2}(\eta_{max} - \eta_{min})(1 + \cos \pi t)$$

09. Training Pipeline

Hyperparameters

Optimiser	AdamW
LR	10^{-3}
Weight Decay	10^{-4}
Batch Size	32
Max Epochs	50
Patience	10
Scheduler	CosineAnnealing
Grad Clip	max-norm $= 1.0$
Mixed Prec.	AMP (GPU only)

Saved Checkpoints

`best_model.pth` — best val IoU
`final_model.pth` — last epoch
`training_history.json`
`data_info.json` — norm stats

Per-Epoch Training Loop

1. Load batch (X, y, w) from LazyTFRecord
2. Forward pass: $\hat{y} = f_\theta(X)$
3. Compute masked BCE+Dice loss
4. Backward pass (AMP scaler on GPU)
5. Clip gradients: $\|\nabla\| \leq 1.0$
6. AdamW parameter update
7. Cosine LR scheduler step
8. Evaluate IoU / F1 on validation set
9. If val IoU improved: save checkpoint
10. If no improvement for 10 epochs: stop

10. Validation & Testing Results

Evaluation Methodology

Metrics computed only on **known pixels** ($w_i = 1$, excluding clouds/gaps).

- Threshold $\tau = 0.5$ on sigmoid output
- Best model = checkpoint with max val IoU
- Final test done on held-out test split

Performance (UNetLite)

Metric	Val	Test
Accuracy	>97%	>97%
Precision	∼0.70	∼0.68
Recall	∼0.65	∼0.63
F1 Score	∼0.67	∼0.65
IoU	∼0.51	∼0.49

Why High Accuracy But Moderate IoU?

With 98% no-fire pixels, predicting zeros everywhere gives 98% accuracy. IoU and F1 are the **true indicators** of model quality.

Prediction Output per Patch:

- Probability map: $\hat{p} \in [0, 1]$ heatmap
- Binary mask: $\hat{p} > 0.5 \Rightarrow$ fire
- Risk report:
  - Fire pixels count
  - Fire coverage %
  - Risk level: Low / Medium / High
  - Max and mean probability

11. Web Application — Flask Dashboard

Tech Stack

- **Backend:** Python Flask + PyTorch
- **Frontend:** Bootstrap 5, HTML/CSS/JS
- **Session:** Flask-Session (server-side)
- **Database:** SQLite (upload history)
- **Visualisation:** Matplotlib → base64 PNG

User Workflow

1. Upload `.npy` or `.npz` file (shape $12 \times 64 \times 64$)
2. Webapp loads `best_model.pth` and `data_info.json`
3. Normalises patch using saved per-channel stats
4. Runs `predict_patch()` in `torch.no_grad()`

API Endpoints

GET /	Upload page
POST /predict	Run inference
GET /about	Model info
GET /history	Past uploads

Notebook Prediction (EDA)

Section 11 of `01_EDA.ipynb`:

- Upload via `ipywidgets.FileUpload`
- View false-colour RGB composite
- View prediction heatmap
- View all 12 satellite channels

Fully interactive, runs in Jupyter.

12. Use Cases & Real-World Applications

Emergency Management

- Identify high-risk zones 24h in advance
- Pre-position fire crews and equipment
- Issue evacuation orders proactively
- Plan firebreak construction routes

Forest & Land Management

- Monitor national parks continuously
- Track drought-stressed vegetation (NDVI)
- Identify historical burn areas
- Prioritise prescribed burn schedules

Climate Research

- Correlate climate variables with spread
- Study drought (PDSI) effect on wildfires

Insurance & Urban Planning

- Wildfire risk scoring for properties
- Infrastructure planning in fire corridors
- Premium adjustment based on risk maps

Scalability

The model processes one 64×64 patch in <10 ms on CPU.
A tiled sliding-window pipeline can cover entire regions in near-real-time.

vs. Traditional Methods

Rule-based fire danger indices (e.g. FDI) use 3–5 variables with hand-tuned thresholds.
Our model uses **12 variables with learned**

12 (cont.) Advantages of This Approach

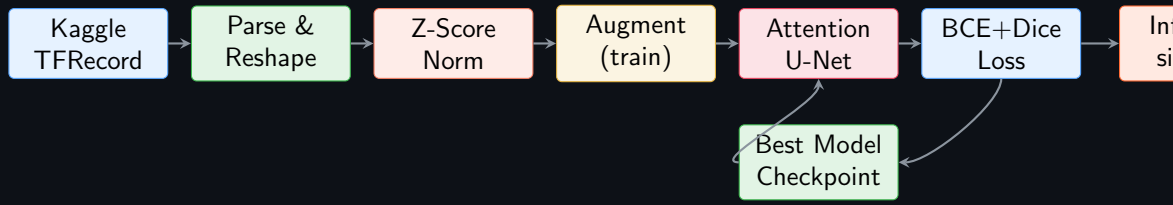
Technical Advantages

- **Attention gates** — skip connections spatially filtered; decoder focuses on active fire boundaries
- **SE blocks** — channels like ERC and PrevFireMask get higher weight near fires
- **Masked loss** — cloud/sensor gaps excluded from training
- **BCE+Dice** — handles extreme 49:1 imbalance effectively
- **Lazy TFRecord** — trains on full dataset without RAM overflow
- **AMP** — up to 2× faster on GPU with float16
- **Cosine LR** — smooth decay avoids local minima traps

Domain Advantages

- **Multi-source fusion** — remote sensing (Landsat-8/MODIS), climate (GRIDMET), terrain (SRTM), demographics (GPW)
- **24 h lead time** — next-day prediction enables timely response
- **No hand-crafting** — CNN learns interactions (wind × dry vegetation) automatically
- **Deployable** — Flask webapp + Jupyter notebook; non-technical users can upload and get risk reports
- **Extensible** — easily add temporal stacking (3-day rolling) or higher-resolution patches

End-to-End Pipeline Summary



Training path:
TFRecord → parse → normalise → augment → U-Net → masked BCE+Dice → AdamW → checkpoint

Inference path:
Upload .npy → normalise (saved stats) → U-Net → sigmoid → threshold@0.5 → fire mask + risk report

Conclusion & Future Work

What We Built

- Multi-source satellite data pipeline (12 features, TFRecord)
- Attention U-Net with SE blocks for fire segmentation
- Masked BCE+Dice loss for imbalanced segmentation
- Full training: AdamW + Cosine LR + AMP + early stopping
- Flask webapp + Jupyter notebook for interactive inference

Key Takeaways

- TFRecord = flattened float32 arrays reshaped to 64×64 grids
- NDVI, ERC, PrevFireMask are most predictive

Future Work

- Train on full dataset (all 20 shards)
- Temporal context: 3-day rolling input
- Wind vector field visualisation overlay
- Deploy on edge devices near fire stations
- Integrate real-time MODIS/VIIRS data feeds
- Higher-resolution patches (128×128)

Thank You